

ML System Design – A Complete Guide (2026)

A Complete Guide

6 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/ml-system-design/>

1. Introduction

What is ML System Design?

ML System Design is the practice of architecting production-grade machine learning systems that reliably deliver predictions at scale. It encompasses data pipelines, training infrastructure, model serving, monitoring, and the operational processes that keep an ML system healthy over time.

Unlike building a model in a notebook — where the goal is to maximize accuracy on a static dataset — ML system design focuses on the end-to-end lifecycle: how does data flow from source to feature to prediction to feedback?

Why ML System Design Matters

- **Production failures are system failures, not model failures:** Most ML projects fail due to data pipeline issues, training-serving skew, monitoring gaps, or infrastructure problems — not because the model's AUC was too low.
- **Scale changes everything:** A notebook model serving 10 requests/day looks nothing like a production system handling 10K QPS with 200ms latency SLAs.
- **ML systems are living systems:** Data distributions shift, user behavior changes, models degrade. Production ML requires continuous monitoring, retraining, and adaptation.
- **Interdisciplinary:** ML system design sits at the intersection of data engineering, software engineering, DevOps, and data science.

ML Systems vs Traditional Software Systems

Aspect	Traditional Software	ML Systems
Behavior	Deterministic, rule-based	Probabilistic, data-driven
Testing	Assert expected output	Evaluate metrics (AUC, precision) on holdout data
Failures	Exceptions, crashes, bugs	Silent quality degradation, drift, stale features
Monitoring	Uptime, latency, errors	+ prediction distribution, feature drift, model accuracy
Dependencies	Libraries, databases, services	+ Training data, features, model artifacts, ground truth labels
Rollback	Deploy previous version	Model rollback + feature rollback + possibly data rollback

2026 State of ML Systems

- Foundation models (LLMs) served via specialized inference engines (vLLM, TGI) with continuous batching.
- Feature stores are standard infrastructure — offline for training, online for serving.
- Automated retraining pipelines triggered by drift detection, not just schedules.
- MLOps platforms (MLflow, W&B, Kubeflow) integrated into CI/CD for model deployments.
- A/B testing and canary deployments are table stakes for model rollouts.

Exercise: Think of an ML-powered feature you use (e.g., YouTube recommendations, spam filter, facial recognition). List three things that could go wrong in production that are not related to model accuracy. For each, suggest a way to detect it.

2. Core Objectives

Every ML system design balances five core objectives. Your design choices prioritize some over others depending on the use case.

1. Scalability

The ability to handle growing data volumes, request rates, and model complexity. Measured by throughput (requests/second, predictions/hour) and data throughput (GB/hour through the pipeline).

- **Horizontal scaling:** Add more servers for inference.
- **Distributed training:** Spread training across GPUs.
- **Partitioned data:** Shard feature stores and model replicas.

2. Latency

End-to-end response time from request to prediction. Critical for real-time applications.

Application	Latency Target
Real-time fraud detection	<100ms
Recommendation serving	<200ms
Chatbot response	<1s
Batch predictions	Minutes–hours

Latency is budgeted across components: network → feature fetch → inference → post-processing.

3. Reliability

Measured by uptime percentage and prediction quality consistency. Includes fault tolerance (graceful degradation when components fail) and accuracy stability (model

quality does not degrade over time).

4. Adaptability

The ability to respond to changing conditions: data distribution shifts, new user behaviors, business requirement changes.

- **Model retraining:** Scheduled (daily) or triggered (drift detected).
- **Feature adaptability:** Add/remove features without retraining everything.
- **Configurability:** Feature flags, model version routing, A/B experiments.

5. Explainability & Fairness

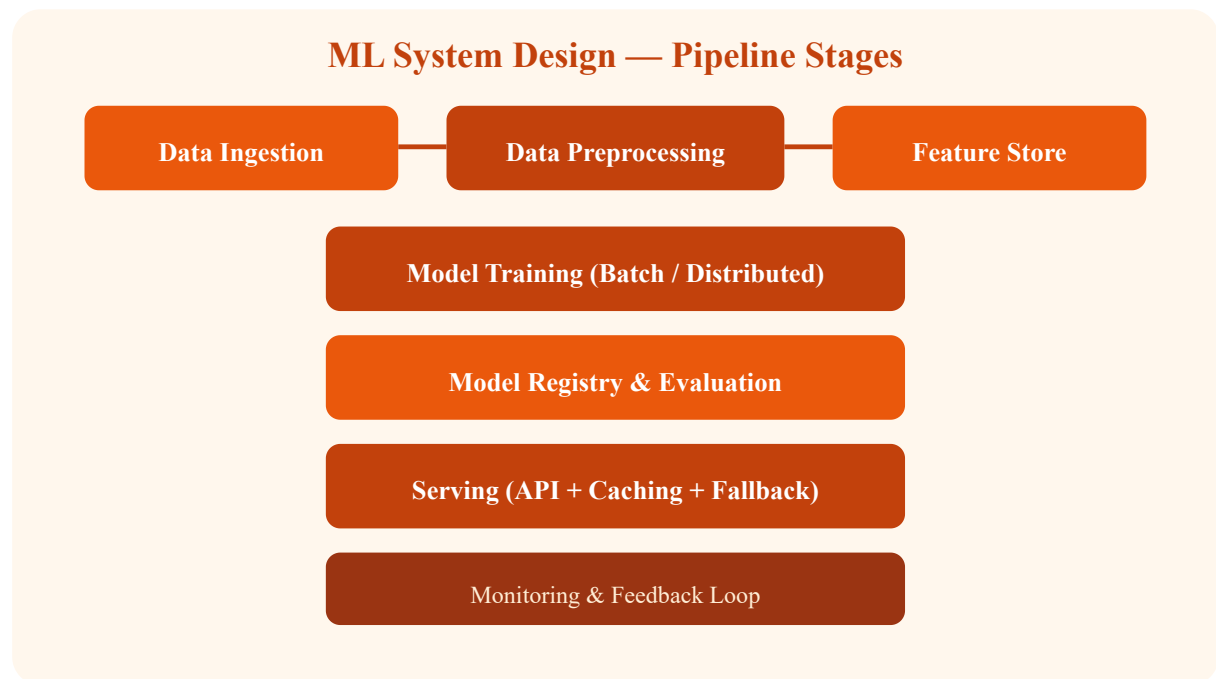
Increasingly non-negotiable for regulated industries. Explainability means understanding *why* a model made a particular prediction. Fairness means ensuring the model does not discriminate against protected groups.

- **Explainability tools:** SHAP, LIME, feature importance, attention visualization.
- **Fairness metrics:** Demographic parity, equal opportunity, disparate impact.
- **Bias detection:** Monitor prediction rates across demographic groups.

```
# Example: fairness monitoring
Group A (majority): approval rate = 72%
Group B (minority): approval rate = 68%
Disparity: 4% → acceptable (threshold: 5%)
If disparity > 5%: alert for investigation
```

Exercise: Rank the five objectives for each scenario: a) an ML-powered medical diagnosis system, b) a real-time ad serving system, c) a batch ML job that generates weekly reports. Explain the reasoning behind each ranking.

3. Stages of ML Systems



Every ML system can be understood as three major stages. Each stage has its own infrastructure requirements, failure modes, and design patterns.

Stage 1: Data Pipeline

The data pipeline is responsible for getting raw data from sources into a usable form for training and serving.

- **Ingestion:** Collect data from event streams (Kafka), databases, APIs, or file uploads.
- **Validation:** Schema validation, anomaly detection, data quality checks.
- **Preprocessing:** Cleaning, normalization, feature engineering, splitting (train/val/test).
- **Storage:** Raw data in data lake (S3), processed features in feature store (offline + online).
- **Versioning:** Dataset versioning for reproducibility (DVC, LakeFS).

```
# Data pipeline failure modes
```

- Schema change in source (new field, removed field, type change)
- Missing values increase beyond threshold
- Late-arriving data breaks training-serving consistency
- Data quality issue (duplicates, outliers) degrades model silently

Stage 2: Training Pipeline

Transforms features and labels into a trained model artifact.

- **Feature loading:** Query offline feature store for training dataset.
- **Training:** Single-node or distributed training across GPUs.
- **Evaluation:** Compute metrics on validation and test sets.
- **Registration:** Log metrics, parameters, and model artifact to model registry.
- **Promotion:** If metrics exceed champion model, stage for deployment.

```
# Training pipeline success criteria
```

- Training completes within SLA (e.g., under 4 hours)
- Validation metrics meet thresholds (e.g., AUC > 0.85)
- No data leakage between train and val sets
- Model artifact size within deployment budget

Stage 3: Serving Pipeline

Exposes the trained model for inference requests.

- **Model loading:** Load model weights from registry into memory.
- **Feature serving:** Fetch real-time features from online feature store.
- **Inference:** Run model prediction with cached or live features.
- **Post-processing:** Apply business rules, thresholds, calibration.
- **Monitoring:** Log latency, predictions, features for drift analysis.

Exercise: Draw the three-stage pipeline for a spam detection ML system. For each stage, list: the specific technologies you would use, the data formats flowing through, what could go wrong, and how you would detect failure.

4. Step-by-Step Architecture

A production ML system architecture built step by step, from raw data to deployed prediction.

Step 1: Data Ingestion

Raw data enters the system through batch or streaming paths.

```
# Ingestion architecture  
Batch path: Daily database dump → Airflow → S3 raw bucket  
Streaming path: User events → Kafka → Flink → S3 raw bucket + Feature Store
```

Step 2: Data Storage

Raw data is stored in a data lake (S3, ADLS, GCS). Processed features go to a feature store (Feast, Tecton). Labels are stored in a data warehouse (Snowflake, BigQuery).

Step 3: Preprocessing

Spark or Dask jobs transform raw data into features. The same transformation logic must be used in training AND serving to avoid training-serving skew.

```
# Consistent transformation (shared code)  
def compute_features(raw):  
    return {  
        "amount_log": log(raw.amount + 1),  
        "hour_of_day": raw.timestamp.hour,  
        "is_weekend": raw.timestamp.weekday() >= 5,  
        "merchant_risk": lookup_merchant_risk(raw.merchant_id)  
    }
```

Step 4: Training

Training jobs read features from the offline feature store, train models using frameworks (PyTorch, XGBoost, TensorFlow), and log results to the model registry.

Step 5: Model Evaluation & Registration

Models are evaluated against champion metrics. If they win, they are registered and staged for deployment. Approval gates may require manual review.

Step 6: Deployment

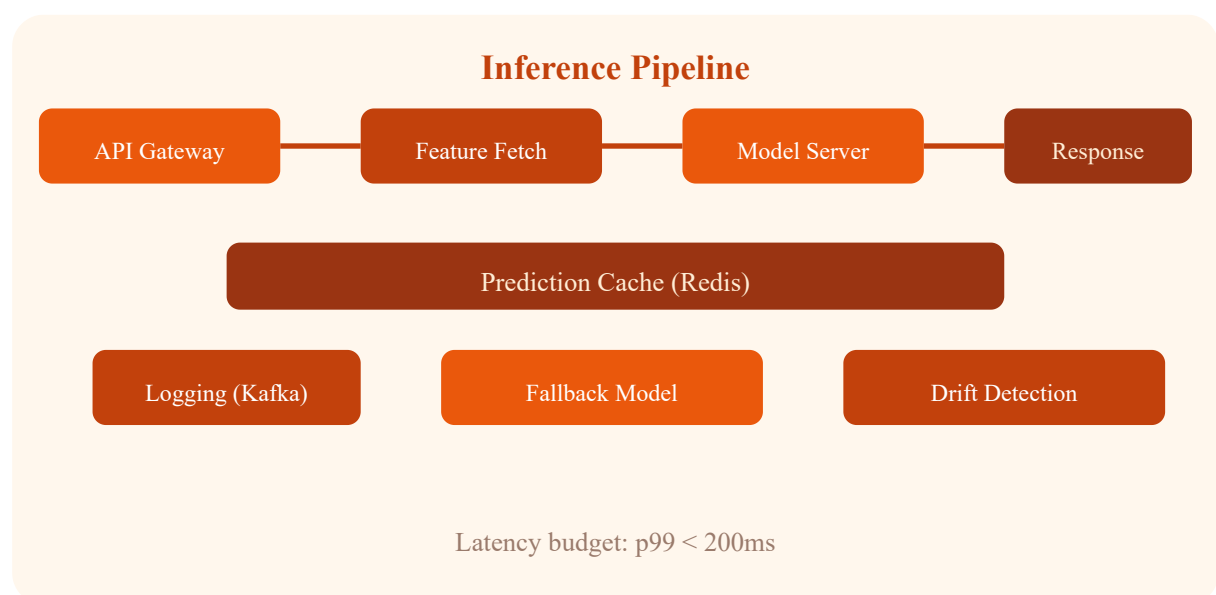
The model is deployed to a serving infrastructure. Common patterns: rolling update, canary deployment, shadow deployment.

Step 7: Serving

Inference requests hit the serving layer: API gateway → feature fetch → model server → response. Predictions are cached, logged, and monitored.

Step 8: Monitoring & Feedback

Predictions and features are logged. Drift detection runs continuously. Ground truth (when available) is collected for accuracy computation. Degradation triggers alerts and retraining.



Exercise: Using the 8-step architecture, design an ML system for detecting fraudulent credit card transactions. For each step, identify: the technology you would use, the data flowing through, the failure mode most likely to occur, and your mitigation strategy.

5. Core Components

Production ML systems are built from reusable components. Mastering each component's design considerations is essential.

Feature Store

A centralized repository for features shared across training and serving.

- **Offline store:** Parquet/AVRO in S3 for batch training. Supports point-in-time queries to avoid data leakage.
- **Online store:** Redis / DynamoDB for sub-millisecond feature lookup during inference.
- **Feature serving API:** gRPC or REST endpoint returning features for entity keys (user_id, product_id).
- **Consistency:** Eventual consistency is typically acceptable (features are minutes stale). Strong consistency for critical features.

Model Registry

A versioned catalog of models with metadata:

- Model binary / weights / ONNX file
- Training metrics (AUC, precision, recall)
- Hyperparameters and training configuration
- Dataset version used for training
- Deployment status (staging, production, archived)
- Approval gate history

Inference API

The interface between the ML system and its consumers.

```
# Inference API design
POST /predict
{
  "features": {"amount": 299.99, "merchant_id": "amzn", ...},
  "request_id": "req_abc",
  "model_version": "v3.2" # optional, defaults to champion
}

Response:
{
  "prediction": 0.12,
  "model_version": "v3.2",
  "latency_ms": 142,
  "explanation": {"amount": 0.05, "merchant_risk": 0.07}
}
```

Feedback Loop

The mechanism for collecting ground truth and improving the system over time.

- **Explicit feedback:** User ratings, thumbs up/down, corrections.
- **Implicit feedback:** Clicks, conversions, dwell time.
- **Outcome labels:** Transaction confirmed fraud/legit, recommendation clicked/purchased.
- **Delay handling:** Ground truth may arrive hours or days after prediction. Store predictions until labels are available.

```
# Feedback store schema
request_id (PK) | features | prediction | ground_truth | label_time
req_abc        | {...}   | 0.12      | 0.0          | 2026-06-15T10:35:00
req_def        | {...}   | 0.87      | 1.0          | 2026-06-15T11:00:00
```

Exercise: Design the core components for an ML system that predicts customer churn. Specify: a) what features go in the feature store, b) what metadata the model registry should store, c) the inference API request/response format, d) how the feedback loop collects churn labels (which may take 30+ days to confirm).

6. Batch vs Real-Time Systems

Batch ML Systems

Batch systems process data in scheduled intervals. All predictions are pre-computed and stored.

Aspect	Batch
Frequency	Hourly, daily, weekly
Latency	Minutes to hours
Compute	Spark, Airflow, scheduled jobs
Storage	Pre-computed predictions in DB or cache
Use cases	Daily recommendations, churn predictions, credit scoring

```
# Batch ML pipeline (daily)
At 2 AM:
1. Load yesterday's features (offline store)
2. Load ground truth for evaluation
3. Retrain model if drift detected or scheduled
4. Batch predict for all active users
5. Write predictions to predictions table
6. Applications read pre-computed predictions
```

Real-Time ML Systems

Real-time systems make predictions on-demand with sub-second latency.

Aspect	Real-Time
Trigger	Request-driven (user action, API call)
Latency	Milliseconds to seconds
Compute	Inference servers (Triton, TorchServe, vLLM)
Storage	Online feature store (Redis), prediction cache
Use cases	Fraud detection, real-time recommendations, ad serving

Lambda Architecture

Processes data through both batch (comprehensive, high latency) and speed (approximate, low latency) layers, merging results in the serving layer.

- **Batch layer:** Complete data, accurate results, higher latency.
- **Speed layer:** Recent data, approximate results, low latency.
- **Serving layer:** Merges batch and speed results for final output.

Kappa Architecture

Everything is treated as a stream. Batch is replayed stream processing. Simpler infrastructure but requires robust stream processing (Kafka + Flink).

Choosing Your Approach

Scenario	Recommendation
You need instant predictions (fraud, ads)	Real-time
Predictions can be pre-computed (daily feed)	Batch
Mix of historical and real-time data	Lambda
Simple pipeline, stream-native team	Kappa

Exercise: A music streaming app needs two ML features: a) a daily "Discover Weekly" playlist for each user, b) a "Next Song" recommendation that updates instantly based on the current song and user behavior. Design the system — which features use batch, which use real-time, and how do they share infrastructure (feature store, model registry)?